

UT2 - Práctica 1: Configuración y Depuración en Java

Fecha: 21/01/2026 | Módulo: Entornos de Desarrollo - 1º DAW

Actividad 5: Configuración del IDE para Java

He descargado y descomprimido el archivo "DepurarJava.zip" del aula virtual. He abierto la carpeta "Depurar1" con Visual Studio Code.

Instalación de requisitos para Java

Para poder ejecutar archivos Java en VS Code he necesitado instalar:

1. JDK (Java Development Kit)

- Descargado desde visualcode JDK Downloader - JDK 25
- Instalación marcando "Add to PATH"
- Verificación en terminal: `java -version`

2. Extension Pack for Java (Microsoft)

- Instalado desde el marketplace de VS Code
- Incluye: Language Support, Debugger, Maven, Test Runner

```
path...
[2026-01-28T20:32:59.285Z] [INFO]: Updating jdk.jdkhome settings...
[2026-01-28T20:32:59.285Z] [INFO]: Finishing up installation...
[2026-01-28T20:32:59.285Z] [INFO]: Copying JDK to installation path successfu
[2026-01-28T20:32:59.303Z] [INFO]: JDK installation completed successfully
[2026-01-28T20:32:59.407Z] [ERROR]: Nbcode Process is null
```



Depuración de Principal.java

Código:

```
1
2  ✓ public class Principal {
3
4      Run | Debug
5      public static void main(String[] args) {
6          int i = 0;
7          while (i < 5) {
8              System.out.println(i);
9              i++;
10         }
11         System.out.println(i); // ¿Qué valor tiene i aquí?
12     }
13
14 }
```

Ejecución normal:

El programa imprime:

```
AlumGS@DESKTOP-VEFAJ8R MINGW64 ~/Desktop/tareaets
$ cd c:\\Users\\AlumGS\\Desktop\\tareaets ; /usr/bin/env C:\\Program\\ Files\\
Java\\jre1.8.0_471\\bin\\java.exe -cp C:\\Users\\AlumGS\\AppData\\Roaming\\Cod
e\\User\\workspaceStorage\\69141e7931333264a4f30424afca0d03\\redhat.java\\jdt_
● ws\\tareaets_99672297\\bin Principal
0
1
2
3
4
5
```

Depuración paso a paso

He colocado un breakpoint en la línea 4 (`int i = 0`) y he ejecutado en modo debug.

Seguimiento de la variable `i`:

Iteración	Valor de <code>i</code>	Condición <code>i < 5</code>	Acción
Inicio	0	<code>0 < 5 = true</code>	Entra al bucle
1	<code>0 → imprime → 1</code>	<code>1 < 5 = true</code>	Continúa
2	<code>1 → imprime → 2</code>	<code>2 < 5 = true</code>	Continúa
3	<code>2 → imprime → 3</code>	<code>3 < 5 = true</code>	Continúa
4	<code>3 → imprime → 4</code>	<code>4 < 5 = true</code>	Continúa
5	<code>4 → imprime → 5</code>	<code>5 < 5 = false</code>	Sale del bucle

¿Qué valor tiene `i` al final del bucle?

La variable `i` tiene el valor **5**.

Explicación: El bucle se ejecuta mientras `i < 5` sea verdadero. Cuando `i` llega a 5, la condición `5 < 5` es falsa y sale del bucle. Por eso, en la línea 8 (`System.out.println(i)`), imprime el valor 5.

Comando java -version

He ejecutado en la terminal integrada de VS Code:

```
java -version
```

Resultado:

```
PS C:\Users\AlumGS\Desktop\tareaets> java -version
java version "1.8.0_471"
Java(TM) SE Runtime Environment (build 1.8.0_471-b09)
Java HotSpot(TM) 64-Bit Server VM (build 25.471-b09, mixed mode)
PS C:\Users\AlumGS\Desktop\tareaets>
```

¿Qué hace cada elemento?

- **JRE (Java Runtime Environment):** Entorno de ejecución que permite ejecutar programas Java compilados (.class)
- **JDK (Java Development Kit):** Kit de desarrollo que incluye el compilador `javac` y herramientas para crear programas
- **JVM (Java Virtual Machine):** Máquina virtual que ejecuta el bytecode Java

¿Cuadra con los lenguajes compilados vistos en teoría?

Sí. Java es un lenguaje **compilado e interpretado** (híbrido):

1. El código fuente `.java` se **compila** con `javac` a bytecode `.class`
2. El bytecode es **interpretado** por la JVM en cada sistema operativo
3. Esto permite que Java sea **multiplataforma** (write once, run anywhere)

Actividad 6: Compilando y generando el Bytecode

Para distribuir aplicaciones Java, generalmente se proporcionan los archivos `.class` (bytecode) en lugar del código fuente.

Compilación con javac

He ejecutado en la terminal el siguiente comando:

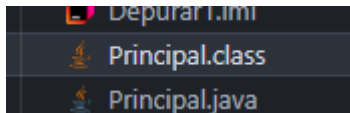
```
bash
```

```
javac Principal.java
```

¿Qué ha pasado?

El comando ha compilado el código fuente y ha generado el archivo Principal.class en la misma carpeta.

```
PS C:\Users\AlumGS\Desktop\tareaets\Depurar1> & "C:\Users\AlumGS\Desktop\test\oracleJdk-25\bin\javac.exe" Principal.java
PS C:\Users\AlumGS\Desktop\tareaets\Depurar1>
```



¿Se te ha generado el Bytecode?

Sí, se ha generado el archivo **Principal.class** que contiene el bytecode.

¿Puedes abrir el fichero con un editor de texto?

Sí puedo abrirlo, pero el contenido no es legible para humanos:

```

E p % E
    @ @ @
    @ @ @ java/lang/Object @ <init> @ ()V @

@ java/lang/System @ Out @ Ljava/io/PrintStream;
    @ @ @
    @ @ @ java/io/PrintStream @ println @ (I)V @ @ Principal @ @ Code @ @ LineNumberTable @ @ main @ @ ([Ljava/lang/String;)
V @
StackMapTable @
SourceFile @ Principal.java ! @ @ @ @ @ @ @ @ @ * . @ ± @ @ @ @ @ U @ @ @ @ < @ ¢ @ ^ @ @
± @ @ @ @ @ @ @ @ @ @ @ @
±
@
@ @ @ @ @ @ @ @ @

```

El bytecode está en formato binario diseñado para ser ejecutado por la JVM, no para ser leído por personas.

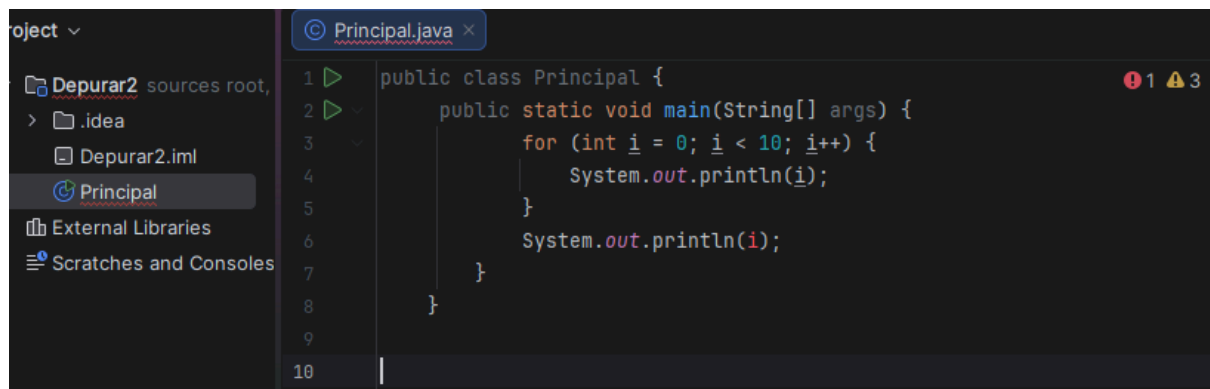
En IntelliJ si es legible:

```
//  
// Source code recreated from a .class file by IntelliJ IDEA  
// (powered by Fernflower decompiler)  
//  
  
public class Principal {  
    public static void main(String[] var0) {  
        int var1;  
        for(var1 = 0; var1 < 5; ++var1) {  
            System.out.println(var1);  
        }  
  
        System.out.println(var1);  
    }  
}
```

Actividad 7: Depurar2

He abierto la carpeta "Depurar2" desde IntelliJ.

Código:



¿Puedes ejecutarlo?

No, el código genera un error de compilación.

Error:

```
urar2: build failed At 28/01/: 2 sec, 175 ms
Principal.java 1 error
cannot find symbol variable i :6
java: cannot find symbol
symbol:   variable i
location: class Principal
```

¿A qué se debe el error?

Análisis con depuración:

He comentado la línea problemática (`// System.out.println(i);`) para poder ejecutar el resto del código y depurar paso a paso.

He colocado un breakpoint en la línea 3 (inicio del bucle for).

```
1 public class Principal {
2     public static void main(String[] args) {
3         for (int i = 0; i < 10; i++) {
4             System.out.println(i);
5         }
6         System.out.println(i);
7     }
8 }
9
10
```

Observación de variables:

Durante la depuración, observo que la variable `i` solo existe dentro del bucle for:

- Dentro del for: `i` aparece en el panel de variables con valores 0, 1, 2... 9
- Fuera del for (línea 6): `i` no existe en el panel de variables

```
C:\Users\Alum6S\Desktop\test\oracleJdk-25\bin\java.exe -agentlib:jdwp=transport=dt_socket,address=1
Connected to the target VM, address: '127.0.0.1:55775', transport: 'socket'
0
1
2
3
4
5
6
7
```

En Java, cuando declaras una variable dentro de un bucle `for (int i = 0; ...)`, esa variable tiene ámbito local al bucle. Es decir, solo existe dentro de las llaves `{ }` del `for`.

java

// Aquí i EXISTE

```
public class Principal {  
    public static void main(String[] args) {  
        for (int i = 0; i < 10; i++) {  
            System.out.println(i);  
        }  
    }  
}
```

// Aquí i NO EXISTE

```
    // System.out.println(i);  
}
```

Solución:

Si necesitamos usar `i` fuera del bucle, debemos declararla antes:

java

`int i;` *// Declarar fuera del bucle*

`for (i = 0; i < 10; i++) {`

`System.out.println(i);`

`}`

`System.out.println(i);` *// Ahora sí existe: imprimirá 10*

```
public class Principal {  
    public static void main(String[] args) {  
        int i = 0;  
        while (i < 5) {  
            System.out.println("Dentro del bucle: " + i);  
            i++;  
        }  
  
        // Al salir, i vale 5 porque es el valor que rompió la condición (5 < 5 es falso)  
        System.out.println("Valor final de i: " + i);  
    }  
}
```

Actividad 7: Depurar3

He abierto la carpeta "Depurar3" desde IntelliJ.

Código:

A screenshot of the IntelliJ IDEA code editor. The top bar shows two tabs: 'Depurar3.iml' and 'Principal.java'. The code in 'Principal.java' is as follows:

```
1
2 public class Principal {
3
4     public static void main(String[] args) {
5         String texto = "hola"; //null;
6         int longitud = texto.length();
7         System.out.println("La longitud del texto es: " + longitud);
8     }
9 }
10
11 }
```

¿Puedes ejecutarlo?

Sí, el código se ejecuta correctamente con `texto = "hola"`.

Salida:

La longitud del texto es: 4

A screenshot of a Windows command prompt window. The prompt shows the directory path 'C:\Users\AlumGS\Desktop\test\oracleJ...' and the output of the Java program: 'La longitud del texto es: 4'.

Análisis del código comentado

Observo que en la línea 3 hay un comentario: `//null;`

Esto indica que originalmente la variable podría haber sido:

java

`String texto = null;`

¿Qué pasaría si descomento esa parte?

He modificado el código para probar:

java

```
public class Principal {  
  
    public static void main(String[] args) {  
        String texto = null;  
        int longitud = texto.length();  
        System.out.println("La longitud del texto es: " + longitud);  
    }  
}
```

Error en ejecución:

Exception in thread "main" java.lang.NullPointerException
at Principal.main(Principal.java:6)

```
C:\Users\Alum6S\Desktop\test\oracleJdk-25\bin\java.exe -javaagent:C:\Users\Alum6S\Downloads\idea-2025.3.2.win\lib\idea_rt.jar=49490 -t  
Exception in thread "main" java.lang.NullPointerException: Cannot invoke "String.length()" because "texto" is null  
at Principal.main(Principal.java:6)
```

Depuración paso a paso

He colocado un breakpoint en la línea 6 (`int longitud = texto.length();`).

Con `texto = "hola"`:

- Variable `texto` = "hola"
- `texto.length()` devuelve 4 ✓
- Funciona correctamente

```
args = {String[0]@923} []  
> texto = "hola"  
10 longitud = 4
```

Con `texto = null`:

- Variable `texto` = null (no apunta a ningún objeto)
- `texto.length()` intenta llamar un método sobre null

- **NullPointerException** ✕

```
③ args = {String[0]@923} []  
☰ texto = null
```

```
create breakpoint : Cannot invoke "String.length()" because "texto" is null
```

Explicación:

En Java, **null** significa que la variable no apunta a ningún objeto. Intentar llamar a un método (**.length()**) sobre **null** genera un **NullPointerException** porque no hay objeto sobre el cual ejecutar el método.

Solución: Siempre validar que una variable no sea null antes de usarla:

java

```
public class Principal {  
    public static void main(String[] args) {  
        String texto = null;  
        if (texto != null) {  
            int longitud = texto.length();  
            System.out.println("La longitud del texto es: " + longitud);  
        } else {  
            System.out.println("El texto es null");  
        }  
    }  
}
```

```
public static void main(String[] args) {  
    String texto = "hola";  
    if (texto != null) {  
        int longitud = texto.length();  
        System.out.println("La longitud del texto es: " + longitud);  
    } else {  
        System.out.println("El texto es null");  
    }  
}
```

Actividad 8: Depurar4

Código:

```
public class Principal {  
  
    public static void main(String[] args) {  
        int[] array = {1, 2, 3};  
        int valor = array[3];  
        System.out.println("El valor es: " + valor);  
    }  
}
```

¿Se produce algún error? ¿Por qué?

Sí, error en ejecución:

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 3 out of bounds for length 3  
    at Principal.main(Principal.java:6)
```

Explicación: El array tiene 3 elementos con índices 0, 1, 2. El índice 3 no existe.

Depuración: Al poner breakpoint y ver variables, el array solo tiene posiciones [0], [1], [2].

```
public static void main(String[] args) {    args: []  
    int[] array = {1, 2, 3};    array: [1, 2, 3]  
    int valor = array[3] [ArrayIndexOutOfBoundsException];    array: [1, 2, 3]  
    System.out.println("El valor es: " + valor);  
}
```

Código corregido:

```
public class Principal {  
  
    public static void main(String[] args) {  
        int[] array = {1, 2, 3};  
        int valor = array[2]; // Último elemento válido  
        System.out.println("El valor es: " + valor);  
    }  
}
```

```
C:\Users\AlumGS\Desktop\test\oracleJdk-  
-Dsun.stdout.encoding=UTF-8 -Dsun.std  
El valor es: 3  
  
Process finished with exit code 0
```

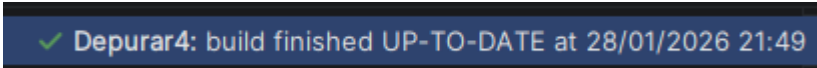
Actividad 9: Build del Proyecto en IntelliJ

Con el código corregido de Depurar4, voy a hacer el Build de la solución.

Proceso de Build

He ido al menú superior de IntelliJ:

- Build → Build Project (o Ctrl+F9)



✓ Depurar4: build finished UP-TO-DATE at 28/01/2026 21:49

¿Se ha generado el Bytecode?

Sí, se ha generado el archivo `Principal.class`.

¿Dónde se ha generado?

IntelliJ crea una carpeta `out` en la raíz del proyecto:

Depurar4/

└─ src/

| └─ Principal.java

└─ out/

└─ production/

└─ Depurar4/

└─ Principal.class

La ruta completa es: `out/production/Depurar4/Principal.class`

